Architecture Decision Record (ADR)

SLE-4: FINAL VIVA & ADR DEFENSE | COURSE: 02AML204 – INTRO TO AI

STUDENT NAME	PRN	DIVISION / CLASS	SUBMISSION DATE
Shrutika Bhosale	25UAM073	SY B.Tech CSE (AI&ML) - A	October 13, 2026

> **ADR Title: Selection of A* Search with Admissible Heuristic over Uninformed Pathfinding Strategies**
>
> Repository: `https://github.com/shrutikabhosle/IAI_SLE2-main`

1. CONTEXT

While developing the navigation and agent pathfinding engine in SLE-1 and profiling its execution in SLE-2, we encountered severe performance bottlenecks when scaling graph searches across complex campus map topographies. The primary challenge was finding the optimal (shortest) path between arbitrary start and goal coordinates without exhausting compute resources or triggering memory overflow. Uninformed search strategies like Breadth-First Search (BFS) and Depth-First Search (DFS) execute blind traversal across the state space, expanding unnecessary nodes in directions moving away from the goal, resulting in unacceptable memory scaling and execution latencies on large inputs.

2. DECISION

We decided to adopt the **A* Search Algorithm** using an admissible Euclidean/Manhattan distance heuristic function as the default search engine implementation for the campus navigation agent.

3. ALTERNATIVES CONSIDERED

- **Alternative 1: Breadth-First Search (BFS):** Evaluated for unweighted graphs. Guarantees the shortest path but suffers from exponential memory consumption $O(b^d)$ as all frontier nodes at the current depth level must be retained in memory simultaneously.
- **Alternative 2: Depth-First Search (DFS):** Assessed for its low linear spatial overhead $O(bm)$. Rejected due to non-optimality, susceptibility to infinite loops in graph structures with cycles, and high path cost variance.
- **Alternative 3: Greedy Best-First Search:** Considered for rapid execution by relying solely on heuristic evaluation $f(n) = h(n)$. Rejected because it fails to account for accumulated path cost $g(n)$, leading to sub-optimal solutions.

4. PERFORMANCE BENCHMARKS (SLE-2 PROFILING DATA)

Algorithm	Nodes Expanded	Max Frontier Size	Execution Time (ms)	Path Cost (Optimal)
Breadth-First Search (BFS)	1,248	312	18.4 ms	42.5 units
Depth-First Search (DFS)	892	84	14.1 ms	78.2 units (Sub-optimal)
A* Search (Manhattan				

5. RATIONALE & JUSTIFICATION

The choice of A* Search is directly justified by empirical metrics gathered during SLE-2 profiling and architectural modularity principles defined in the SLE-3 C4 diagrams:

- **Computational & Memory Efficiency:** As evidenced by our profiling benchmarks, A* reduces the node expansion space by over **85%** compared to BFS (184 nodes vs. 1,248 nodes). The maximum frontier memory footprint dropped from 312 nodes to 28 nodes.
- **Guaranteed Optimality:** By employing an admissible heuristic function where `h(n) ≤ h*(n)`, the evaluation function `f(n) = g(n) + h(n)` mathematically guarantees the discovery of the shortest path, unlike DFS or Greedy search.
- **Architectural Decoupling (C4 Component Alignment):** In our SLE-3 C4 Component Diagram, the search engine was designed as an independent service interface. A* cleanly encapsulates pathfinding logic using a priority queue (`heapq`), decoupling algorithmic execution from graph generation and UI rendering.

6. CONSEQUENCES

Positive Outcomes:

- Achieved sub-5ms pathfinding responses even across complex campus map graph vertices.
- Optimized memory retention during concurrent agent path requests.
- Maintainable codebase with clear segregation between heuristic calculations and state generation.

Trade-offs & Limitations:

- Requires storing all visited nodes in memory (`closed_set`), leading to higher spatial overhead than DFS.
- Pathfinding efficiency remains tightly bounded to the quality and consistency of the distance heuristic.

PART B: SUMMARY OF SLE JOURNEY

SLE-1: Built core state-space graph models, implemented search routines (`bfs_dfs.py`), established repository structures, and recorded the initial AI Contribution log.
SLE-2: Executed performance profiling (`banchmark.py` , `profilesearch.py`) measuring runtime latency, node expansion, and memory scaling across search algorithms.
SLE-3: Formulated comprehensive C4 architecture models (Context, Container, Component, Code level diagrams) detailing interaction flows.
SLE-4: Formalized key architecture choices via this ADR document and defended trade-offs during oral viva.

PART C: AI CONTRIBUTION NOTE

> **AI Tools Used:** ChatGPT / Claude for document formatting and technical review.
>
> **Assistance Rendered:** AI helped structure the standard ADR section headings and refine technical phrasing regarding time/space complexity trade-offs.
>
> **Self-Authored Work:** All algorithm code (`bfs_dfs.py` , `profilesearch.py`), profiling benchmarking, C4 diagrams, profiling metrics analysis, and final decision rationale were designed, implemented, and verified independently by Shrutika Bhosle.